

План собеседования Middle+ Go разработчика

Цель интервью

Оценить, способен ли кандидат уровня Middle+ самостоятельно проектировать и реализовывать production-grade сервисы на Go, принимать инженерные решения, находить компромиссы между простотой и масштабируемостью, разбираться в существующем коде, работать с конкурентностью, базами данных, API, тестированием и эксплуатацией сервисов.

Ожидаемый профиль Middle+

Кандидат уровня Middle+ обычно:

- уверенно пишет идиоматичный Go и понимает стандартную библиотеку;
- умеет проектировать HTTP/gRPC-сервисы, API-контракты и внутренние пакеты;
- понимает goroutines, channels, context, race conditions и базовые паттерны конкурентности;
- работает с SQL/NoSQL, транзакциями, индексами и миграциями;
- пишет unit, integration и table-driven тесты;
- способен диагностировать performance-проблемы и production-инциденты;
- понимает observability: logs, metrics, tracing, alerts;
- умеет обсуждать архитектурные trade-off и защищать технические решения;
- может ревьюить код и быть владельцем части сервиса без постоянного контроля.

Рекомендуемая структура интервью

Общая длительность: 90-120 минут.

Блок	Длительность	Что проверяем
1. Вводная часть и опыт	10 мин	Релевантность опыта, зрелость, зона ответственности
2. Go core и идиоматика	20 мин	Знание языка, стандартной библиотеки, ошибок, интерфейсов
3. Concurrency и context	20 мин	Goroutines, channels, cancellation, races, deadlocks
4. Backend, API и данные	20 мин	HTTP/gRPC, БД, транзакции, надежность
5. Практическая задача	25-35 мин	Код, декомпозиция, тестируемость, edge cases
6. System design light	20 мин	Архитектурное мышление, компромиссы, эксплуатация
7. Поведенческий блок и вопросы кандидата	10 мин	Коммуникация, ownership, работа в команде

1. Вводная часть и опыт

Цели блока

- Понять, какие задачи кандидат реально решал сам.
- Выяснить размер и критичность систем, с которыми он работал.
- Отличить практический production-опыт от поверхностного знакомства с технологиями.

Основные вопросы

1. Расскажите о последнем Go-проекте: какую проблему решал сервис, какая была ваша зона ответственности?
2. Какой самый сложный технический выбор вы делали в этом проекте? Какие альтернативы рассматривали?
3. С какими нагрузками работал сервис: RPS, объем данных, latency, количество пользователей или сообщений?
4. Как была организована разработка: code review, тестирование, CI/CD, релизы, мониторинг?
5. Приведите пример production-инцидента, в расследовании которого вы участвовали. Как нашли причину?
6. Что в Go вам нравится, а что кажется неудобным или ограничивающим?

Сильные сигналы

- Кандидат говорит конкретно: метрики, ограничения, решения, последствия.
- Может объяснить не только что делал, но и почему.
- Осознает стоимость решений: complexity, maintainability, latency, reliability.

Риски

- Описывает опыт общими словами без деталей.
- Не может объяснить архитектуру сервиса, над которым работал.
- Приписывает себе командные решения, но не показывает личный вклад.

2. Go core и идиоматика

Темы для проверки

- packages, visibility, init;
- pointers, values, escape analysis на базовом уровне;
- slices, maps, arrays;
- interfaces и embedding;
- error handling;
- defer, panic, recover;
- generics;
- standard library;
- memory model на практическом уровне.

Вопросы по языку

1. Чем slice отличается от array? Что хранит slice header?
2. Что произойдет при append к slice, если capacity достаточно, и если capacity недостаточно?
3. Почему изменение одного slice может неожиданно изменить другой slice?
4. Можно ли читать из nil map? Можно ли писать в nil map?
5. Что происходит при обращении к отсутствующему ключу map?
6. Почему map в Go не потокобезопасна?
7. Когда стоит использовать pointer receiver, а когда value receiver?
8. Чем interface value с nil concrete value отличается от nil interface?
9. Как Go выбирает, реализует ли тип интерфейс?

10. Для чего нужен empty interface или any? Когда его использование ухудшает дизайн?
11. Как работает defer? В каком порядке выполняются несколько defer?
12. Что будет с аргументами defer: вычисляются сразу или в момент выполнения?
13. Когда допустимо использовать panic/recover в backend-сервисе?
14. Как вы обычно оборачиваете и сравниваете ошибки?
15. Чем errors.Is отличается от errors.As?
16. Когда уместны sentinel errors, а когда лучше typed errors?
17. Что такое zero value и почему это важная идея в Go?
18. Как generics изменили подход к utility-коду? Когда не стоит использовать generics?
19. Какие пакеты стандартной библиотеки вы используете чаще всего?
20. Что такое escape analysis и как она может влиять на производительность?

Практические мини-вопросы

Slice aliasing

```
a := []int{1, 2, 3, 4}
b := a[:2]
b = append(b, 99)
fmt.Println(a)
```

Вопросы:

- Что будет выведено?
- Почему?
- Как избежать неожиданного изменения исходного slice?

Ожидаемое направление ответа: кандидат понимает shared backing array и предлагает copy или full slice expression с ограничением capacity.

Nil interface

```
type MyError struct{}

func (e *MyError) Error() string { return "boom" }

func do() error {
    var err *MyError = nil
    return err
}

fmt.Println(do() == nil)
```

Вопросы:

- Что будет выведено?
- Почему error interface не равен nil?
- Как правильно вернуть отсутствие ошибки?

Ожидаемое направление ответа: interface содержит dynamic type и dynamic value; nil должен возвращаться как untyped nil.

3. Concurrency и context

Темы для проверки

- goroutines lifecycle;
- channels: buffered/unbuffered, close, select;

- sync.Mutex, RWMutex, WaitGroup, Once, Cond;
- atomic operations;
- context cancellation and deadlines;
- race conditions;
- worker pools;
- backpressure;
- graceful shutdown.

Основные вопросы

1. Чем goroutine отличается от OS thread?
2. Что может привести к goroutine leak?
3. Как обнаружить goroutine leak в production?
4. Когда использовать channel, а когда mutex?
5. Что произойдет при чтении из закрытого channel?
6. Что произойдет при записи в закрытый channel?
7. Кто должен закрывать channel: отправитель или получатель?
8. Для чего нужен select с default?
9. Как реализовать fan-out/fan-in?
10. Как ограничить количество одновременно выполняющихся задач?
11. Как корректно остановить worker pool?
12. Как работает context.Context и что нельзя хранить в context?
13. Чем context cancellation отличается от timeout/deadline?
14. Как передавать context в функции и методы?
15. Почему нельзя игнорировать ctx.Done() в долгих операциях?
16. Что такое data race? Как race detector помогает, а где его недостаточно?
17. Когда использовать sync/atomic вместо mutex?
18. Что такое backpressure и как его реализовать?
19. Как сделать graceful shutdown HTTP-сервера?
20. Как бы вы защитили сервис от лавинообразного роста goroutines?

Практический concurrency-сценарий

Нужно обработать 10 000 задач, каждая делает HTTP-запрос к внешнему сервису. Внешний сервис нестабилен, есть rate limit, запросы могут зависать.

Попросите кандидата описать решение:

- ограничение параллелизма;
- context timeout на запрос;
- retry policy с backoff;
- обработка частичных ошибок;
- сбор результатов;
- остановка при отмене;
- метрики и логирование;
- защита от goroutine leaks.

Сильный ответ включает worker pool или semaphore, context propagation, bounded queue, timeouts, idempotency considerations, observability и понятную стратегию ошибок.

4. Backend, API и работа с данными

HTTP/gRPC и API

1. Как вы проектируете REST API для нового ресурса?
2. Какие HTTP-коды вы используете для validation error, conflict, unauthorized, forbidden, not found?
3. Как организовать versioning API?
4. Как валидировать входные данные?
5. Где должна жить бизнес-логика: handler, service, repository?
6. Как вы обрабатываете request timeout?
7. Чем middleware отличается от handler wrapper?
8. Как реализовать idempotency для POST-запроса?
9. Когда выбирать gRPC вместо REST?
10. Какие проблемы возникают при backward-incompatible изменениях API?

Базы данных

1. Как устроена транзакция? Что такое commit и rollback?
2. Какие уровни изоляции транзакций вы знаете?
3. Что такое dirty read, non-repeatable read, phantom read?
4. Как избежать lost update?
5. Как работает optimistic locking?
6. Как выбрать индекс для SQL-запроса?
7. Почему индекс может не использоваться?
8. Как анализировать медленный SQL-запрос?
9. Чем INNER JOIN отличается от LEFT JOIN?
10. Что такое N+1 problem и как ее исправить?
11. Как проектировать миграции без downtime?
12. Как хранить и обрабатывать nullable-поля в Go?
13. Когда использовать SQL, а когда NoSQL?
14. Как организовать pagination: offset vs cursor?
15. Как вы проектируете repository layer?

Reliability

1. Что такое timeout, retry и circuit breaker?
2. Почему retries могут ухудшить инцидент?
3. Что такое idempotency key?
4. Как обрабатывать partial failure?
5. Как проектировать взаимодействие с внешним API?
6. Что логировать при ошибке, а что нельзя логировать?
7. Какие метрики нужны для HTTP-сервиса?
8. Как понять, что сервис деградирует до полной поломки?

5. Практическая задача

Выберите одну задачу в зависимости от фокуса вакансии. Для Middle+ важно не только получить работающий код, но и увидеть ход мыслей, декомпозицию, обработку ошибок и тестируемость.

Вариант А: In-memory rate limiter

Задача: реализовать in-memory rate limiter для API по userID.

Требования:

- лимит N запросов за окно времени T;
- потокобезопасность;
- метод `Allow(userID string) bool`;
- очистка старых записей;
- возможность написать unit tests;
- обсуждение поведения при большом количестве пользователей.

Вопросы после реализации:

1. Какой алгоритм вы выбрали: fixed window, sliding window, token bucket? Почему?
2. Какие edge cases есть на границе окна?
3. Как избежать бесконечного роста map?
4. Как сделать реализацию распределенной?
5. Какие метрики вы бы добавили?

Критерии оценки:

- корректная синхронизация;
- простая и понятная структура данных;
- тесты на границы окна;
- отсутствие глобального состояния;
- понимание ограничений in-memory решения.

Вариант В: Concurrent fetch with timeout

Задача: написать функцию, которая параллельно запрашивает данные по списку URL и возвращает успешные ответы.

Пример сигнатуры:

```
func FetchAll(ctx context.Context, client *http.Client, urls []string, limit int) ([]Result, error)
```

Требования:

- ограничить параллелизм параметром `limit`;
- уважать отмену context;
- не допустить goroutine leaks;
- вернуть частичные результаты;
- корректно обработать HTTP errors и timeouts;
- написать table-driven tests с mock server.

Вопросы после реализации:

1. Как остановятся goroutines при отмене context?
2. Что произойдет, если один URL зависнет?
3. Как сохранить порядок результатов?

4. Как добавить retry?
5. Как логировать ошибки без утечки персональных данных?

Вариант C: Transactional service method

Задача: спроектировать сервисный метод перевода денег между счетами.

Требования:

- проверить существование счетов;
- проверить достаточность средств;
- выполнить debit и credit атомарно;
- защититься от concurrent transfers;
- сохранить audit log;
- вернуть доменные ошибки;
- обсудить тестирование.

Вопросы после реализации:

1. Где должна начинаться транзакция?
2. Какой уровень изоляции выбрать?
3. Как избежать deadlock?
4. Как сделать операцию идемпотентной?
5. Как обработать сбой после commit, но до ответа клиенту?

6. System design light

Сценарий

Спроектируйте сервис уведомлений, который принимает события от других сервисов и отправляет email, SMS и push-уведомления.

Что попросить кандидата раскрыть

1. API приема событий.
2. Валидация и нормализация данных.
3. Очередь или брокер сообщений.
4. Хранение статуса уведомлений.
5. Retry и dead-letter queue.
6. Idempotency и дедупликация.
7. Rate limits внешних провайдеров.
8. Шаблоны уведомлений.
9. Observability.
10. Graceful degradation.
11. Horizontal scaling.
12. Безопасность и персональные данные.

Уточняющие вопросы

1. Что произойдет, если провайдер SMS недоступен 30 минут?
2. Как не отправить одно уведомление дважды?

3. Как гарантировать, что событие не потеряется?
4. Какой `delivery guarantee` выбрать: `at-most-once`, `at-least-once` или `exactly-once`?
5. Какие таблицы или коллекции нужны?
6. Какие индексы понадобятся?
7. Как мониторить задержку доставки?
8. Как отделить критичные уведомления от маркетинговых?
9. Как безопасно хранить шаблоны и пользовательские данные?
10. Как тестировать такую систему?

Сильный ответ

Кандидат не пытается сразу построить чрезмерно сложную систему, а уточняет требования, выделяет критичные потоки, объясняет `trade-off` между простотой, надежностью и стоимостью. В ответе должны появиться `idempotency`, `retries`, `DLQ`, `outbox` или аналогичный паттерн для надежной публикации событий, метрики, `structured logs` и понятная модель данных.

7. Тестирование и качество

Вопросы

1. Как вы обычно пишете `unit tests` в Go?
2. Что такое `table-driven tests` и почему они популярны в Go?
3. Как тестировать код, который зависит от времени?
4. Как тестировать HTTP handler?
5. Как тестировать интеграцию с базой данных?
6. Когда использовать `mocks`, `fakes` и `testcontainers`?
7. Как проверять `race conditions`?
8. Что должно быть в CI pipeline для Go-сервиса?
9. Какой уровень покрытия тестами вы считаете полезным?
10. Как вы относитесь к `snapshot tests`?

Практическое обсуждение

Дайте кандидату небольшой фрагмент плохо тестируемого кода: прямой вызов `time.Now`, глобальный HTTP client, работа с БД внутри handler. Попросите объяснить, как сделать код тестируемым.

Ожидаемые идеи:

- `dependency injection`;
- интерфейсы на границах;
- `clock abstraction`;
- `httptest`;
- `transaction rollback` в интеграционных тестах;
- разделение `handler/service/repository`.

8. Performance и profiling

Вопросы

1. Как понять, что Go-сервис упирается в CPU, память, БД или сеть?
2. Как использовать pprof?
3. Какие профили вы знаете: cpu, heap, goroutine, mutex, block?
4. Что такое allocation и почему лишние allocations важны?
5. Когда стоит оптимизировать код?
6. Как benchmark отличается от unit test?
7. Как написать корректный benchmark в Go?
8. Что может привести к высокому GC pressure?
9. Как искать memory leak в Go?
10. Какие типичные причины latency spikes?

Сильные сигналы

- Кандидат начинает с измерений, а не с догадок.
- Умеет пользоваться pprof, metrics и tracing.
- Понимает, что преждевременная оптимизация может ухудшить дизайн.

9. Observability и эксплуатация

Вопросы

1. Какие structured logs нужны в HTTP-сервисе?
2. Что должно быть в request ID или trace ID?
3. Какие RED/USE-метрики вы знаете?
4. Какие alerts вы бы настроили для backend-сервиса?
5. Чем symptoms-based alerts лучше cause-based alerts?
6. Как логировать ошибки внешних API?
7. Как связаны tracing и logs?
8. Что должно быть в health check?
9. Чем readiness отличается от liveness?
10. Как организовать graceful shutdown в Kubernetes?

10. Security

Вопросы

1. Как хранить пароли пользователей?
2. Почему нельзя логировать access tokens и персональные данные?
3. Чем authentication отличается от authorization?
4. Как устроен JWT? Какие у него риски?
5. Что такое CSRF и когда он актуален?
6. Что такое SQL injection и как от него защищаться?
7. Как валидировать входные данные?
8. Как хранить secrets для сервиса?
9. Как ограничить blast radius при компрометации сервиса?

10. Какие security checks можно добавить в CI?

11. Code review

Формат

Покажите кандидату pull request или фрагмент кода с намеренными проблемами:

- business logic в HTTP handler;
- отсутствие context propagation;
- бесконечные retries;
- игнорирование ошибок;
- race condition;
- нет timeout у HTTP client;
- SQL-запрос без индекса;
- логирование чувствительных данных;
- непроверенные edge cases.

Вопросы

1. Какие проблемы вы видите?
2. Какие замечания обязательны к исправлению, а какие можно оставить как улучшение?
3. Как бы вы сформулировали комментарий в code review?
4. Какие тесты нужно добавить?
5. Что можно упростить без изменения поведения?

Сильный ответ

Кандидат приоритизирует риски, не заикливаясь на стиле, объясняет влияние проблемы на пользователей или эксплуатацию, предлагает конкретные исправления и умеет коммуницировать без агрессии.

12. Поведенческий блок

Вопросы

1. Расскажите о случае, когда вы не согласились с техническим решением команды. Что сделали?
2. Когда вы в последний раз сильно упростили решение? Что помогло увидеть простоту?
3. Приведите пример, когда вы ошиблись в production. Как исправляли последствия?
4. Как вы принимаете решение между быстрым fix и правильным refactoring?
5. Как вы помогли менее опытному разработчику?
6. Что для вас хороший code review?
7. Как вы действуете, если задача плохо сформулирована?
8. Как вы сообщаете менеджеру или команде о техническом риске?
9. Что вы делаете, если не успеваете закончить задачу в ожидаемый срок?
10. Как вы поддерживаете качество в условиях давления на сроки?

13. Матрица оценки

Оценивайте каждый блок по шкале 1-4.

Балл	Интерпретация
1	Знания фрагментарные, много критичных пробелов, нужен постоянный контроль
2	Уровень ближе к Middle-, решает типовые задачи, но слабо видит риски
3	Уверенный Middle/Middle+, самостоятельно решает production-задачи
4	Сильный Middle+/Senior-, системно мыслит, предвидит проблемы, усиливает команду

Компетенции

Компетенция	Вес	Что считать хорошим результатом
Go core и идиоматика	20%	Понимает язык, ошибки, interfaces, slices/maps, стандартную библиотеку
Concurrency	20%	Уверенно работает с goroutines, channels, context, races
Backend и данные	20%	Проектирует API, транзакции, надежную работу с внешними системами
Практическое кодирование	20%	Пишет простой, корректный, тестируемый код
Архитектура и эксплуатация	15%	Видит trade-off, observability, reliability, scaling
Коммуникация и ownership	5%	Ясно объясняет решения, принимает ответственность

Decision guide

- Strong hire: в большинстве блоков 3-4, нет критичных пробелов в concurrency, данных и production-надежности.
- Hire: стабильные 3, отдельные слабые зоны компенсируются опытом и обучаемостью.
- Lean no hire: много ответов на уровне теории, слабая практика с context, транзакциями или тестами.
- No hire: не может объяснить базовые конструкции Go, не видит race conditions, игнорирует ошибки и эксплуатационные риски.

14. Красные флаги

- Не понимает nil interface и error handling в Go.
- Не умеет объяснить lifecycle goroutine и риски goroutine leaks.
- Предлагает unbounded goroutines для внешних запросов.
- Игнорирует context cancellation.
- Использует panic для обычных бизнес-ошибок.
- Не пишет тесты или считает их необязательными.
- Не умеет объяснить транзакции и конкурентное обновление данных.
- Считает retries универсальным решением без timeout и backoff.
- Не различает authentication и authorization.
- Не может рассказать о собственных ошибках и выводах.

15. Вопросы кандидата к интервьюеру

Оставьте 5-10 минут на вопросы кандидата. Хорошие вопросы кандидата тоже являются сигналом зрелости.

На что обратить внимание:

- спрашивает о домене, архитектуре, нагрузке, команде;
- интересуется качеством процессов, on-call, observability;
- уточняет ожидания от роли и зоны ответственности;
- задает вопросы о техническом долге и планах развития.

16. Итоговый шаблон заметок интервьюера

Кандидат:

Дата:

Интервьюеры:

Краткое резюме:

Go core:

Оценка 1-4:

Заметки:

Concurrency:

Оценка 1-4:

Заметки:

Backend/API/DB:

Оценка 1-4:

Заметки:

Практическая задача:

Оценка 1-4:

Заметки:

System design:

Оценка 1-4:

Заметки:

Коммуникация и ownership:

Оценка 1-4:

Заметки:

Сильные стороны:

Риски:

Рекомендация:

Strong hire / Hire / Lean no hire / No hire

Аргументация:

17. Подробные вопросы и ответы для интервьюера

Этот раздел можно использовать как ключ к интервью. Ответы не требуют дословного совпадения: сильный кандидат объясняет идею своими словами, приводит примеры из production и видит ограничения решения.

17.1 Go core и идиоматика

1. Чем slice отличается от array? Что хранит slice header?

Ожидаемый ответ: array в Go имеет фиксированную длину, и длина является частью типа: `[3]int` и `[4]int` - разные типы. Slice - это легковесное представление над backing array. Slice header содержит указатель на первый элемент доступного участка массива, длину `len` и емкость `cap`.

Что важно услышать: при передаче slice в функцию копируется header, но underlying array остается общим. Поэтому изменение элементов внутри функции видно снаружи, а append может либо использовать тот же массив, либо выделить новый.

Пример хорошего объяснения: если `s := []int{1,2,3}` передать в функцию и сделать `s[0] = 10`, вызывающий увидит изменение. Если внутри функции сделать `s = append(s, 4)`, изменение длины локально, но элементы backing array могут быть затронуты.

2. Что происходит при append к slice при достаточной и недостаточной capacity?

Ожидаемый ответ: если capacity достаточно, append добавляет элементы в существующий backing array и возвращает новый slice header с увеличенной длиной. Если capacity недостаточно, runtime выделяет новый backing array, копирует старые элементы, добавляет новые и возвращает slice, указывающий на новый массив.

Практический вывод: результат append всегда нужно присваивать: `s = append(s, x)`. Нельзя полагаться, что старый slice указывает на новые данные.

Красный флаг: кандидат считает, что append всегда меняет исходный slice или всегда выделяет новый массив.

3. Почему изменение одного slice может неожиданно изменить другой slice?

Ожидаемый ответ: несколько slice могут ссылаться на один backing array. Если один slice меняет элемент или делает append в пределах общей capacity, другой slice может увидеть эти изменения.

Пример:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[2:]
a = append(a, 99)
fmt.Println(base, b)
```

append в a может перезаписать элемент base[2], потому что capacity a позволяет писать дальше в тот же массив.

Как избежать: использовать copy, slices.Clone, full slice expression `a := base[:2:2]`, чтобы ограничить capacity, или проектировать код так, чтобы ownership slice был очевиден.

4. Можно ли читать из nil map? Можно ли писать в nil map?

Ожидаемый ответ: читать из nil map можно: вернется zero value типа значения и `ok == false` при comma-ok форме. Писать в nil map нельзя: будет panic assignment to entry in nil map.

Правильная практика: map нужно инициализировать через `make(map[K]V)` или литерал `map[K]V{}` перед записью.

5. Что происходит при обращении к отсутствующему ключу map?

Ожидаемый ответ: возвращается zero value для типа значения. Чтобы отличить отсутствующий ключ от реально сохраненного zero value, используют форму `value, ok := m[key]`.

Пример: для `map[string]int` отсутствующий ключ даст 0, поэтому без ok нельзя понять, был ли ключ или значение действительно равно нулю.

6. Почему map в Go не потокобезопасна?

Ожидаемый ответ: стандартная map не синхронизирует внутреннюю структуру. Одновременная запись или одновременное чтение и запись приводят к data race и могут закончиться panic concurrent map read and map write.

Как исправлять: защищать map через `sync.Mutex/sync.RWMutex`, использовать ownership через одну goroutine и channels, либо `sync.Map` для специфичных сценариев с большим числом read-mostly операций или независимыми ключами.

Сильный ответ: кандидат уточняет workload. `sync.Map` не является универсальной заменой `map+mutex` и может быть хуже по читаемости и производительности.

7. Когда использовать `pointer receiver`, а когда `value receiver`?

Ожидаемый ответ: `pointer receiver` нужен, если метод меняет состояние объекта, если структура большая и копирование дорого, если нужно сохранить identity, или если часть методов уже использует `pointer receiver` и нужен единый `method set`. `Value receiver` подходит для маленьких `immutable-like` типов, где копирование безопасно и ожидаемо.

Важная деталь: `method set` отличается для `T` и `*T`. Тип `T` имеет методы с `value receiver`, а `*T` имеет и `value`, и `pointer receiver`. Это влияет на реализацию интерфейсов.

8. Чем `interface value` с `nil concrete value` отличается от `nil interface`?

Ожидаемый ответ: `interface` в Go концептуально содержит `dynamic type` и `dynamic value`. `Interface` равен `nil` только если оба компонента `nil`. Если вернуть `(*MyError)(nil)` как `error`, `dynamic type` будет `*MyError`, а `dynamic value` `nil`, поэтому `interface` не равен `nil`.

Правильная практика: при отсутствии ошибки возвращать обычный `nil`, а не `typed nil`. Если функция создает `typed error pointer`, нужно явно проверять его до возврата.

9. Как Go выбирает, реализует ли тип интерфейс?

Ожидаемый ответ: интерфейсы реализуются неявно: тип реализует интерфейс, если его `method set` содержит все методы интерфейса с совпадающими сигнатурами. Никакого `implements` писать не нужно.

Сильная деталь: кандидат понимает разницу `method set` для `value` и `pointer receiver` и может объяснить, почему `T` может не реализовать интерфейс, а `*T` реализует.

10. Для чего нужен `any` и когда он ухудшает дизайн?

Ожидаемый ответ: `any` - alias для `interface{}`. Он полезен на границах с динамическими данными: JSON, логирование, generic containers, совместимость с API. Но чрезмерное использование `any` убирает статическую типизацию, переносит ошибки в runtime и усложняет контракт функции.

Хороший критерий: если тип можно выразить явно или через небольшой интерфейс поведения, лучше не использовать `any`.

11. Как работает `defer` и в каком порядке выполняются несколько `defer`?

Ожидаемый ответ: `defer` откладывает вызов функции до выхода из текущей функции. Несколько `defer` выполняются в обратном порядке: LIFO. Аргументы `deferred`-вызова вычисляются в момент объявления `defer`, а не в момент исполнения.

Практика: `defer` часто используют для `Unlock`, `Close`, `rollback`, `recovery middleware`. В горячих циклах `defer` может быть нежелателен, если создает лишнюю стоимость или откладывает освобождение ресурса до конца всей функции.

12. Когда допустимо использовать `panic/recover` в backend-сервисе?

Ожидаемый ответ: `panic` не должен быть способом обработки обычных бизнес-ошибок. Его можно использовать для `truly unrecoverable` состояний, `programmer errors`, невозможной инициализации, или внутри инфраструктурного `middleware`, который восстанавливает процесс, логирует `panic` и возвращает 500.

Сильный ответ: `recover` должен быть на границе запроса/goroutine. `Panic` в отдельной `goroutine` не поймается `recover` в другой `goroutine`.

13. Как оборачивать и сравнивать ошибки?

Ожидаемый ответ: для добавления контекста используют `fmt.Errorf("...: %w", err)`. Для проверки цепочки ошибок используют `errors.Is`, для извлечения конкретного типа - `errors.As`. Сравнение `err == someErr` работает только без `wrapping` или при прямом `sentinel error`.

Пример: repository может вернуть `ErrUserNotFound`, service оборачивает `get user: %w`, handler через `errors.Is(err, ErrUserNotFound)` мапит ошибку в 404.

14. Чем `errors.Is` отличается от `errors.As`?

Ожидаемый ответ: `errors.Is` отвечает на вопрос "есть ли в цепочке эта ошибка или ошибка, считающаяся эквивалентной". `errors.As` отвечает на вопрос "есть ли в цепочке ошибка данного типа, и можно ли ее извлечь".

Пример: `errors.Is(err, sql.ErrNoRows)` для `sentinel`. `errors.As(err, &validationErr)` для `typed error` с полями.

15. Когда уместны `sentinel errors`, а когда `typed errors`?

Ожидаемый ответ: `sentinel errors` удобны для небольшого набора стабильных состояний: `not found`, `conflict`, `unauthorized`. `Typed errors` лучше, когда нужно передать структурированные детали: поле валидации, внешний статус, `retryable flag`, `operation id`.

Риск: публичные `sentinel errors` становятся частью API пакета. Их нужно проектировать как контракт, а не как временную деталь.

16. Что такое `zero value` и почему это важно?

Ожидаемый ответ: `zero value` - значение по умолчанию для типа: `0`, `false`, `""`, `nil`, `struct` с `zero`-полями. Go поощряет типы, которые полезны или безопасны в `zero value`: `sync.Mutex`, `bytes.Buffer`.

Дизайн-вывод: хороший Go API часто позволяет создать объект без сложного конструктора, если это не нарушает инварианты.

17. Когда использовать `generics`, а когда нет?

Ожидаемый ответ: `generics` полезны для типобезопасных алгоритмов и контейнеров, где логика одинакова для разных типов: `set`, `map/filter helpers`, `constraints` для `numeric` операций. Не стоит использовать `generics`, если интерфейс поведения проще, если типов мало, или если `generic`-код ухудшает читаемость.

Сильный ответ: кандидат не использует `generics` для бизнес-абстракций без реальной пользы.

18. Что такое `escape analysis`?

Ожидаемый ответ: `compiler analysis`, который решает, может ли значение жить на `stack` или должно быть перемещено в `heap`. Значение `escape`-ится, если его `lifetime` выходит за рамки `stack frame` или `compiler` не может доказать безопасность `stack allocation`.

Практический смысл: `heap allocations` увеличивают нагрузку на GC. Но вручную оптимизировать нужно после измерений: `benchmarks`, `pprof`, `go build -gcflags=-m`.

17.2 Concurrency и context

19. Чем `goroutine` отличается от `OS thread`?

Ожидаемый ответ: `goroutine` - легковесная единица выполнения, управляемая Go runtime scheduler. Многие `goroutines` мультиплексируются на меньшее число `OS threads`. `Goroutine` стартует с небольшим `stack`, который может расти. `OS thread` тяжелее и управляется ОС.

Сильный ответ: кандидат понимает, что легковесность не означает бесплатность: `goroutines` потребляют память, `scheduler time` и могут создавать нагрузку на внешние ресурсы.

20. Что приводит к `goroutine leak`?

Ожидаемый ответ: `goroutine leak` возникает, когда `goroutine` больше не нужна, но не может завершиться. Частые причины: ожидание чтения/записи в `channel` без получателя/отправителя, отсутствие реакции на `context cancellation`, бесконечный `retry-loop`, зависший внешний вызов без `timeout`.

Как предотвращать: передавать context, закрывать каналы с понятным ownership, использовать timeouts, bounded queues, WaitGroup, errgroup, проверять lifecycle в тестах.

21. Как обнаружить goroutine leak в production?

Ожидаемый ответ: смотреть рост числа goroutines через runtime metrics, pprof goroutine dump, heap/profile, traces, correlating с RPS и ошибками. Если число goroutines растет после завершения нагрузки и не возвращается, это сигнал leak.

Сильный ответ: кандидат умеет читать goroutine stack traces и связывать их с конкретным blocking point.

22. Когда использовать channel, а когда mutex?

Ожидаемый ответ: mutex хорошо подходит для защиты shared state и коротких критических секций. Channel подходит для передачи ownership, coordination, pipeline, fan-out/fan-in, сигналов завершения. Не стоит использовать channel только потому, что это Go, если обычный mutex проще.

Критерий: если задача - "защитить map", чаще нужен mutex. Если задача - "передать работу worker-ам и собрать результат", channel подходит естественно.

23. Что происходит при чтении из закрытого channel?

Ожидаемый ответ: чтение из закрытого channel немедленно возвращает zero value типа элемента. В comma-ok форме `v, ok := <-ch`, `ok` будет false, когда channel закрыт и буфер пуст.

Деталь: если buffered channel закрыли, сначала будут прочитаны оставшиеся элементы, затем начнут возвращаться zero value и `ok=false`.

24. Что происходит при записи в закрытый channel?

Ожидаемый ответ: запись в закрытый channel вызывает panic. Поэтому закрывать channel должен тот, кто владеет отправкой, обычно sender или координатор sender-ов.

Красный флаг: кандидат предлагает закрывать channel со стороны receiver без гарантии, что никто больше не пишет.

25. Для чего нужен select с default?

Ожидаемый ответ: default делает select неблокирующим. Это полезно для попытки отправки/получения без ожидания, polling, drop policy при переполненной очереди. Но в цикле `for { select { default: } }` можно получить busy loop и загрузить CPU.

Хорошая практика: если нужен цикл, часто добавляют blocking case, timer/ticker или backoff.

26. Как реализовать fan-out/fan-in?

Ожидаемый ответ: fan-out: несколько worker goroutines читают задачи из общего input channel. Fan-in: результаты нескольких worker-ов собираются в один output channel. Нужны WaitGroup для закрытия output после завершения всех worker-ов и context для отмены.

Сильный ответ: кандидат учитывает backpressure через bounded channels и не забывает закрыть output только после завершения всех отправителей.

27. Как ограничить количество одновременно выполняющихся задач?

Ожидаемый ответ: использовать worker pool фиксированного размера, semaphore на buffered channel, errgroup с limit, или rate limiter. Выбор зависит от того, есть ли очередь задач, нужны ли результаты и как обрабатывается отмена.

Пример: buffered channel `sem := make(chan struct{}, limit)`; перед запуском работы отправляем token, после завершения освобождаем token через defer.

28. Как корректно остановить worker pool?

Ожидаемый ответ: закрыть input channel, чтобы worker-ы завершили range; или отменить context, чтобы worker-ы прекратили ожидание/работу. Координатор ждет worker-ы через WaitGroup.

Output channel закрывается после завершения всех worker-ов.

Сильный ответ: кандидат объясняет, как избежать отправки в закрытый channel и как завершить обработку частичных результатов.

29. Как работает context.Context и что нельзя хранить в context?

Ожидаемый ответ: context переносит deadline, cancellation signal и request-scoped values через границы API. Его нужно передавать первым параметром функции. В context не хранят optional parameters, большие объекты, mutable state, database handles, logger как обязательную зависимость, если это ломает явный контракт.

Практика: context values подходят для request id, trace id, auth claims, если они действительно request-scoped.

30. Почему нельзя игнорировать ctx.Done() в долгих операциях?

Ожидаемый ответ: если операция не слушает context, она продолжит потреблять ресурсы после отмены запроса, timeout клиента или shutdown. Это ведет к goroutine leaks, лишней нагрузке и медленному завершению сервиса.

Пример: worker, который делает HTTP requests, должен создавать request через `http.NewRequestWithContext`.

31. Что такое data race?

Ожидаемый ответ: data race возникает, когда две goroutines одновременно обращаются к одной памяти, хотя бы одна операция - запись, и между ними нет synchronization. Поведение программы становится неопределенным с точки зрения Go memory model.

Как искать: `go test -race`, code review, pprof/block/mutex profiles. Race detector помогает, но проверяет только выполненные paths, поэтому не доказывает отсутствие races.

32. Когда использовать sync/atomic вместо mutex?

Ожидаемый ответ: atomic подходит для простых независимых операций над числами/указателями: counters, flags, copy-on-write pointers. Mutex лучше для сложных инвариантов, нескольких полей, map/slice, операций read-modify-write над структурой.

Риск: atomic-код легко сделать формально race-free, но логически неверным. Для Middle+ важно выбирать читаемость и корректность.

33. Что такое backpressure?

Ожидаемый ответ: backpressure - механизм, который не дает producer-ам бесконечно генерировать работу быстрее, чем consumers могут ее обработать. Реализация: bounded queues, rate limiting, semaphore, отказ с 429/503, drop policy, circuit breaker.

Сильный ответ: кандидат связывает backpressure с защитой памяти, внешних зависимостей и latency.

34. Как сделать graceful shutdown HTTP-сервера?

Ожидаемый ответ: поймать SIGTERM/SIGINT, перестать принимать новые запросы через `http.Server.Shutdown(ctx)`, дать текущим запросам deadline на завершение, остановить background workers, закрыть подключения к БД/очередям после завершения работы, экспортировать readiness=false до остановки.

Важная деталь: Shutdown использует context с timeout. Если timeout истек, соединения закрываются принудительно.

17.3 Backend, API и данные

35. Как проектировать REST API для нового ресурса?

Ожидаемый ответ: начать с доменной модели и операций над ресурсами, определить URL, методы, request/response schemas, status codes, validation errors, idempotency, pagination,

authorization, versioning. API должен быть стабильным контрактом, а не отражением внутренней структуры БД.

Пример: GET /users/{id}, POST /users, PATCH /users/{id}. Ошибки возвращаются в едином формате с machine-readable code и human-readable message.

36. Какие HTTP-коды использовать для типовых ошибок?

Ожидаемый ответ: 400 Bad Request для синтаксически неверного запроса, 422 Unprocessable Entity для валидного JSON с доменной validation error, 401 Unauthorized для отсутствующей/невалидной аутентификации, 403 Forbidden для запрета при известной личности, 404 Not Found, 409 Conflict, 429 Too Many Requests, 500 для unexpected server errors, 503 для временной недоступности.

Сильный ответ: кандидат не раскрывает internal error клиенту, но логирует details с correlation id.

37. Где должна жить бизнес-логика: handler, service или repository?

Ожидаемый ответ: handler отвечает за transport: parsing, validation на уровне формата, auth context, mapping errors to HTTP. Service содержит бизнес-правила и транзакционные use cases. Repository отвечает за persistence и SQL/DB details.

Практический критерий: бизнес-правило должно тестироваться без HTTP и желательно без реальной БД, если это не integration test.

38. Как реализовать idempotency для POST-запроса?

Ожидаемый ответ: клиент отправляет idempotency key. Сервер сохраняет ключ, fingerprint запроса, статус выполнения и результат. Повторный запрос с тем же ключом и тем же fingerprint возвращает тот же результат. Если fingerprint отличается - ошибка conflict.

Важные детали: атомарное сохранение ключа и результата, TTL для ключей, защита от гонок, учет пользователя/tenant в scope ключа.

39. Когда выбирать gRPC вместо REST?

Ожидаемый ответ: gRPC хорош для internal service-to-service communication, строгих контрактов protobuf, streaming, низкой latency, code generation, polyglot environments. REST проще для публичных API, браузеров, дебага через curl и широкого ecosystem.

Сильный ответ: выбор зависит от клиентов, инфраструктуры, observability, backward compatibility и требований к streaming.

40. Что такое транзакция?

Ожидаемый ответ: транзакция - группа операций с ACID-свойствами: atomicity, consistency, isolation, durability. commit фиксирует изменения, rollback отменяет их. Транзакция нужна, когда несколько изменений должны произойти как единое целое.

Практика в Go: context-aware BeginTx, defer rollback, явный commit, аккуратная обработка ошибки commit.

41. Какие уровни изоляции транзакций вы знаете?

Ожидаемый ответ: Read Uncommitted, Read Committed, Repeatable Read, Serializable. В PostgreSQL Read Uncommitted фактически работает как Read Committed. Более высокий уровень уменьшает аномалии, но может увеличить блокировки, serialization failures и стоимость.

Сильный ответ: кандидат говорит не только названия, но и какие аномалии предотвращаются и как это влияет на retry транзакций.

42. Что такое dirty read, non-repeatable read, phantom read?

Ожидаемый ответ: dirty read - чтение незакоммиченных данных другой транзакции.

Non-repeatable read - повторное чтение той же строки в одной транзакции дает другой результат из-за commit другой транзакции. Phantom read - повторный запрос по условию возвращает другой набор строк из-за вставок/удалений другой транзакции.

43. Как избежать lost update?

Ожидаемый ответ: использовать row-level locks (`SELECT ... FOR UPDATE`), atomic update with condition (`UPDATE accounts SET balance=balance-? WHERE id=? AND balance>=?`), optimistic locking через version column, или serializable transaction с retry.

Сильный ответ: кандидат учитывает deadlocks, порядок блокировок и retry policy.

44. Как работает optimistic locking?

Ожидаемый ответ: у записи есть версия или `updated_at`. При обновлении запрос включает старую версию: `UPDATE table SET ..., version=version+1 WHERE id=? AND version=?`. Если `affected rows = 0`, значит запись изменилась конкурентно, нужно повторить чтение/операцию или вернуть conflict.

45. Как выбрать индекс для SQL-запроса?

Ожидаемый ответ: смотреть WHERE, JOIN, ORDER BY, GROUP BY, selectivity, cardinality и реальные query plans через `EXPLAIN ANALYZE`. Композитный индекс должен учитывать порядок колонок и pattern запроса.

Важная деталь: индекс ускоряет чтение, но замедляет записи и занимает место. Индексы нужны под реальные запросы, а не "на всякий случай".

46. Почему индекс может не использоваться?

Ожидаемый ответ: низкая selectivity, функция над индексируемой колонкой, несовпадение типов, leading wildcard в LIKE, устаревшая статистика, planner считает seq scan дешевле, неправильный порядок колонок в composite index.

47. Что такое N+1 problem и как исправлять?

Ожидаемый ответ: N+1 возникает, когда после одного запроса списка для каждого элемента делается отдельный запрос деталей. Исправление: JOIN, batch query `WHERE id IN (...)`, preloading, dataloader pattern, изменение API/агрегации.

Сильный ответ: кандидат думает о лимитах batch size и размере результата.

48. Как проектировать миграции без downtime?

Ожидаемый ответ: использовать expand/contract pattern. Сначала добавить backward-compatible schema changes, затем задеплоить код, который пишет/читает новое поле, выполнить backfill батчами, переключить чтение, потом удалить старое поле отдельным релизом.

Риски: долгие locks, default values на больших таблицах, backfill без throttling, несовместимость старой и новой версии сервиса.

49. Как хранить nullable-поля в Go?

Ожидаемый ответ: варианты: `sql.NullString` и аналоги, pointer fields, custom nullable-типы, nullable-типы ORM/driver. Выбор зависит от различия между "отсутствует", "null" и zero value.

Сильный ответ: кандидат учитывает JSON serialization и доменную модель, а не просто технический тип БД.

50. Offset vs cursor pagination?

Ожидаемый ответ: offset pagination проста, но на больших данных становится медленной и нестабильной при вставках/удалениях. Cursor pagination использует stable sort key и cursor, лучше для больших списков и realtime changes, но сложнее для произвольного перехода на страницу.

Практика: нужен deterministic ordering, например `(created_at, id)`, чтобы избежать пропусков и дублей.

17.4 Reliability и внешние зависимости

51. Что такое timeout, retry и circuit breaker?

Ожидаемый ответ: timeout ограничивает время ожидания операции. Retry повторяет временно неуспешную операцию. Circuit breaker временно прекращает вызовы к нестабильной зависимости, чтобы дать ей восстановиться и защитить свой сервис.

Сильный ответ: эти механизмы должны использоваться вместе с idempotency, backoff, jitter, retry budget и observability.

52. Почему retries могут ухудшить инцидент?

Ожидаемый ответ: retries увеличивают нагрузку на уже деградирующую зависимость, могут вызвать retry storm, дублировать side effects и увеличить latency. Без backoff и лимитов retries часто превращают частичный сбой в полный.

Правильный подход: retry только для retryable errors, ограниченное число попыток, exponential backoff with jitter, timeout на всю операцию, idempotency key для side-effect операций.

53. Что такое partial failure?

Ожидаемый ответ: ситуация, когда часть операции успешна, а часть нет: один downstream ответил, другой нет; запись в БД прошла, публикация события нет; часть batch обработана. Distributed systems должны проектироваться с учетом partial failure.

Паттерны: transactions там, где возможно, outbox, sagas, compensating actions, idempotent consumers, reconciliation jobs.

54. Как проектировать взаимодействие с внешним API?

Ожидаемый ответ: явные timeouts, retries с backoff, circuit breaker, rate limiting, idempotency для side effects, typed client, contract tests, обработка статусов, structured logging без секретов, metrics по latency/errors, fallback или graceful degradation.

17.5 Практические задачи: ожидаемые решения

55. In-memory rate limiter: какой алгоритм выбрать?

Ожидаемый ответ: для простой задачи подойдет fixed window, но у него есть burst на границе окна. Sliding window точнее, но сложнее и дороже. Token bucket хорошо контролирует среднюю скорость и допускает ограниченный burst. Для API часто выбирают token bucket или sliding window в зависимости от требований.

Корректная реализация: хранить состояние по userID, защищать map mutex-ом или шардировать, использовать injectable clock для тестов, периодически очищать неактивных пользователей, не держать lock во время долгих операций.

Ограничение: in-memory limiter не работает корректно при нескольких инстансах без sticky sessions или внешнего shared storage вроде Redis.

56. Concurrent fetch with timeout: как должна выглядеть архитектура?

Ожидаемый ответ: создать context с общим timeout или принять внешний context, ограничить параллелизм worker pool/semaphore, каждый request создавать через `http.NewRequestWithContext`, результаты отправлять в channel или защищенную структуру, дождаться worker-ов, закрыть result channel, вернуть частичные результаты и агрегированную ошибку.

Важные edge cases: `limit <= 0`, пустой список URL, отмена context до старта, зависший сервер, не-2xx статус, сохранение порядка результатов, отсутствие goroutine leaks.

57. Transactional money transfer: как решить безопасно?

Ожидаемый ответ: начать транзакцию на уровне service/use case, заблокировать счета в детерминированном порядке по id через `SELECT ... FOR UPDATE`, проверить существование и баланс, выполнить debit/credit, записать audit log, commit. Доменные ошибки отделить от инфраструктурных.

Сильные детали: idempotency key для повторного запроса клиента, retry при deadlock/serialization failure, decimal/integer minor units вместо float, audit log в той же транзакции, outbox для события после commit.

17.6 Тестирование

58. Что такое table-driven tests?

Ожидаемый ответ: тесты, где набор кейсов описан таблицей структур, а один test loop прогоняет функцию на каждом кейсе. Это снижает дублирование и делает edge cases явными.

Хорошая практика: использовать `t.Run(name, func(t *testing.T) {...})`, не забывать capture переменной в старых версиях Go/при параллельных тестах, проверять и happy path, и ошибки.

59. Как тестировать код, зависящий от времени?

Ожидаемый ответ: не вызывать `time.Now()` глубоко в бизнес-логике напрямую. Передавать clock interface/function, использовать fixed time в тестах, контролируемый fake clock для timers/tickers.

Риск: тесты со sleep обычно flaky и медленные.

60. Как тестировать HTTP handler?

Ожидаемый ответ: использовать `httptest.NewRequest`, `httptest.ResponseRecorder`, подменять service dependency mock/fake-ом, проверять status code, headers, body, error mapping. Для интеграционного уровня можно поднимать `httptest.Server`.

61. Как тестировать интеграцию с БД?

Ожидаемый ответ: использовать отдельную тестовую БД, миграции, transaction rollback per test или очистку данных, testcontainers/docker при возможности, fixtures/factories, проверять реальные constraints и SQL behavior.

Сильный ответ: кандидат разделяет unit tests repository mocks и integration tests с реальной БД.

62. Когда использовать mocks, fakes и testcontainers?

Ожидаемый ответ: mocks подходят для проверки взаимодействий на границах, но ими легко закрепить implementation details. Fakes часто лучше для доменной логики. Testcontainers/реальная инфраструктура нужны, когда важно поведение драйвера, SQL, брокера, транзакций.

17.7 Performance и profiling

63. Как понять, где bottleneck: CPU, память, БД или сеть?

Ожидаемый ответ: начать с метрик: latency breakdown, CPU, memory, GC, DB query duration, connection pool stats, network errors. Затем использовать pprof, tracing, logs с correlation id. Без измерений оптимизация будет гаданием.

64. Как использовать pprof?

Ожидаемый ответ: подключить net/http/pprof или собрать профиль программно, снять cpu/heap/goroutine/mutex/block profiles, открыть через go tool pprof, смотреть top, list, web/flamegraph. Профилировать нужно под репрезентативной нагрузкой.

65. Что может привести к высокому GC pressure?

Ожидаемый ответ: большое число short-lived allocations, лишние conversions `[]byte/string`, создание объектов в горячих циклах, неограниченные buffers, крупные временные структуры, чрезмерное логирование/JSON encoding.

Исправление: измерить allocations benchmark-ами и heap profile, переиспользовать buffers осторожно, уменьшить промежуточные объекты, stream processing вместо загрузки всего в память.

66. Как искать memory leak в Go?

Ожидаемый ответ: в Go leak часто означает удержание ссылок или goroutine leak. Сравнить heap profiles во времени, смотреть goroutine dump, проверять caches без eviction, slices, которые держат большой backing array, global maps, channels/workers.

17.8 Observability и эксплуатация

67. Какие structured logs нужны HTTP-сервису?

Ожидаемый ответ: request id/trace id, method, path/route pattern, status, latency, user/tenant id при допустимости, error code, dependency name/status/latency для downstream errors. Нельзя логировать пароли, токены, персональные данные без необходимости.

68. Какие RED/USE-метрики вы знаете?

Ожидаемый ответ: RED для сервисов: Rate, Errors, Duration. USE для ресурсов: Utilization, Saturation, Errors. Для HTTP нужны RPS, error rate, latency percentiles, in-flight requests. Для БД - pool usage, query latency, errors.

69. Readiness vs liveness?

Ожидаемый ответ: liveness отвечает, жив ли процесс и нужно ли его перезапустить. Readiness отвечает, готов ли инстанс принимать трафик. При shutdown readiness должна стать false до остановки обработки.

Риск: если liveness зависит от временной недоступности БД, Kubernetes может начать перезапускать здоровые, но временно деградировавшие поды.

17.9 Security

70. Как хранить пароли пользователей?

Ожидаемый ответ: никогда не хранить plaintext. Использовать password hashing algorithms с солью и work factor: bcrypt, scrypt, Argon2id. Соль должна быть уникальной, pepper можно хранить отдельно в secret manager. Нужно поддерживать rehash при изменении параметров.

71. Authentication vs authorization?

Ожидаемый ответ: authentication подтверждает, кто пользователь. Authorization решает, что этому пользователю разрешено. Пользователь может быть аутентифицирован, но не иметь доступа к ресурсу.

72. Как устроен JWT и какие риски?

Ожидаемый ответ: JWT состоит из header, payload claims и signature. Риски: long-lived tokens, невозможность простого revoke без state, неправильная проверка alg/audience/issuer/expiration, хранение секретных данных в payload, утечка токена.

Практика: короткий TTL access token, refresh flow, проверка iss, aud, exp, key rotation, HTTPS, безопасное хранение на клиенте.

73. Как защищаться от SQL injection?

Ожидаемый ответ: использовать parameterized queries/prepared statements, не конкатенировать user input в SQL, валидировать identifiers отдельно через allowlist, ограничивать права DB user. ORM не гарантирует защиту, если разработчик вставляет raw SQL небезопасно.

17.10 Code review

74. Как ревьюить PR с business logic в handler?

Ожидаемый ответ: указать, что handler смешивает transport и доменную логику, из-за чего код сложнее тестировать и переиспользовать. Предложить вынести use case в service, оставить в handler parsing/auth/error mapping. Попросить unit tests на service и handler tests на HTTP mapping.

Хороший комментарий: "Давай вынесем расчет лимита в service: так мы сможем протестировать бизнес-правила без HTTP и не будем дублировать их при появлении gRPC endpoint".

75. Как приоритизировать замечания в code review?

Ожидаемый ответ: сначала correctness, security, data loss, race conditions, observability, backward compatibility. Затем maintainability и тестируемость. Стиль и вкусовые предпочтения должны быть автоматизированы линтерами или помечены как non-blocking.

17.11 System design: сервис уведомлений

76. Какой базовый дизайн предложить?

Ожидаемый ответ: API принимает событие, валидирует его и сохраняет durable record. Через outbox или транзакционную публикацию событие попадает в брокер. Worker-ы отправляют email/SMS/push через provider clients. Статусы и попытки сохраняются в БД. Для retry используется delayed queue/backoff, для исчерпанных попыток - DLQ.

Компоненты: ingress API, DB, outbox publisher, message broker, workers per channel/provider, template service, status storage, metrics/logs/traces, admin/reconciliation tooling.

77. Как не отправить уведомление дважды?

Ожидаемый ответ: использовать idempotency key/event id, unique constraint на (tenant_id, event_id, channel), idempotent consumer, хранить provider message id, дедуплицировать retries. При at-least-once доставке нужно проектировать consumers как idempotent.

78. Как гарантировать, что событие не потеряется?

Ожидаемый ответ: сохранить событие в durable storage в рамках транзакции с outbox record. Отдельный publisher читает outbox и публикует в брокер, пометая опубликованное. Если publisher падает, он продолжает с неопубликованных записей. Брокер должен подтверждать запись, consumer - ack после успешной обработки.

Сильный ответ: кандидат признает, что exactly-once в распределенных системах обычно заменяется at-least-once + idempotency.

79. Что делать, если SMS-провайдер недоступен 30 минут?

Ожидаемый ответ: circuit breaker открывается, retries идут с backoff и jitter, задачи остаются в очереди или delayed retry storage, критичные уведомления можно переключить на fallback provider, некритичные деградировать. Нужны alerts по error rate, queue lag, delivery latency.

80. Какие метрики нужны сервису уведомлений?

Ожидаемый ответ: accepted events rate, validation errors, queue lag, send attempts, success/failure by channel/provider, retry count, DLQ size, delivery latency percentiles, provider latency/status, template rendering errors, worker saturation.

17.12 Поведенческий блок: что считать сильным ответом

81. Кандидат не согласился с техническим решением команды. Что слушать в ответе?

Сильный ответ: кандидат сначала понял контекст и ограничения, сформулировал риски, предложил альтернативы, обсудил trade-off, принял командное решение после дискуссии и помог его реализовать. Важно отсутствие позиции "я один был прав, остальные не понимали".

82. Кандидат ошибся в production. Что слушать?

Сильный ответ: конкретное описание инцидента, impact, timeline, mitigation, root cause, follow-up actions. Кандидат не перекладывает вину, говорит о системных улучшениях: tests, alerts, rollout strategy, runbook, feature flag.

83. Как кандидат выбирает между quick fix и refactoring?

Сильный ответ: оценивает impact, срочность, риск регрессии, blast radius, наличие тестов. В инциденте сначала безопасная стабилизация, затем отдельный follow-up на системное исправление. Если quick fix создает долг, он должен быть видимым и запланированным.

84. Как кандидат помогает менее опытному разработчику?

Сильный ответ: задает вопросы, объясняет reasoning, дает небольшие самостоятельные задачи, делает pairing, пишет понятные review comments, делится контекстом. Не забирает задачу полностью и не унижает за ошибки.

17.13 Расширенный блок по базам данных

Этот блок стоит использовать, если роль предполагает самостоятельную работу с PostgreSQL/MySQL, проектирование схемы, оптимизацию запросов, миграции и разбор production-проблем с данными. Вопросы можно задавать выборочно: для Middle+ важно не знать все названия наизусть, а уметь объяснять причинно-следственные связи и выбирать безопасное решение.

85. Что происходит внутри БД при выполнении SQL-запроса?

Ожидаемый ответ: в общем виде запрос проходит parsing, semantic analysis, rewrite, planning/optimization и execution. Planner выбирает план на основе статистики, доступных индексов, оценочной стоимости чтения страниц, фильтрации, join strategy и сортировок. Executor выполняет выбранный план и возвращает строки клиенту.

Объяснение: важно, что БД не "просто использует индекс, если он есть". Она сравнивает стоимость разных вариантов. Если таблица маленькая или условие выбирает большую долю строк, sequential scan может быть дешевле index scan. Поэтому оптимизация начинается с EXPLAIN/EXPLAIN ANALYZE, а не с добавления случайных индексов.

Сильный ответ: кандидат упоминает статистику, cardinality/selectivity, join algorithms, сортировки и то, что план может измениться после роста данных.

86. Чем EXPLAIN отличается от EXPLAIN ANALYZE?

Ожидаемый ответ: EXPLAIN показывает предполагаемый план и оценочные стоимости без выполнения запроса. EXPLAIN ANALYZE реально выполняет запрос и показывает фактическое время, количество строк, loops и расхождения между оценками и реальностью.

Объяснение: если estimates сильно отличаются от actual rows, planner может выбрать плохой join или scan. Причиной могут быть устаревшие statistics, коррелированные колонки, неудачный индекс или неравномерное распределение данных.

Осторожность: EXPLAIN ANALYZE для INSERT/UPDATE/DELETE выполняет изменение. В production его нужно использовать аккуратно, иногда внутри rollback-транзакции или на replica/staging.

87. Какие поля в EXPLAIN ANALYZE вы смотрите первыми?

Ожидаемый ответ: actual time, actual rows, loops, разницу estimated rows vs actual rows, тип scan, join type, sort method, heap fetches, buffers при BUFFERS, filter rows removed, наличие temporary files.

Объяснение: медленный запрос часто не в том месте, где самая большая cost estimate, а там, где много loops или резко больше actual rows. Например, nested loop может быть нормальным для десятков строк, но разрушительным для сотен тысяч.

Сильный ответ: кандидат умеет объяснить, почему один node с loops=100000 может давать основную задержку даже при маленьком времени одного loop.

88. Что такое selectivity и cardinality индекса?

Ожидаемый ответ: cardinality - количество уникальных значений в колонке или наборе колонок. Selectivity - насколько условие фильтрует данные: какую долю строк оно оставляет. Чем выше selectivity условия, тем полезнее индекс.

Объяснение: индекс по boolean-полю часто бесполезен, если значения распределены 50/50 и запрос выбирает большую часть таблицы. Но partial index по редкому состоянию, например WHERE status = 'pending', может быть очень полезен.

Пример: для таблицы заказов индекс по user_id обычно полезен, если запросы выбирают заказы одного пользователя из миллионов строк. Индекс по is_deleted, где почти все строки false,

часто не помогает без partial condition.

89. Как работает composite index и почему важен порядок колонок?

Ожидаемый ответ: composite index строится по нескольким колонкам в заданном порядке. Для B-tree эффективно используется leftmost prefix: индекс (tenant_id, created_at, id) хорошо подходит для фильтра по tenant_id и сортировки/диапазона по created_at, но не так полезен для запроса только по created_at без tenant_id.

Объяснение: первая колонка должна обычно соответствовать equality/filter с хорошим сужением или tenant boundary, затем идут колонки диапазона и сортировки. Но порядок зависит от конкретных запросов, распределения данных и условий.

Сильный ответ: кандидат не отвечает универсальным "самую селективную колонку первой" без контекста. Для multi-tenant систем tenant_id часто первым нужен для изоляции данных и pruning, даже если другая колонка более селективна глобально.

90. Чем B-tree, Hash, GIN и GiST индексы отличаются на практике?

Ожидаемый ответ: B-tree - основной индекс для equality, range, ordering. Hash - для equality, но в PostgreSQL используется реже, потому что B-tree обычно достаточно универсален. GIN полезен для массивов, JSONB, full-text search, containment. GiST полезен для геоданных, ranges, similarity и некоторых custom operators.

Объяснение: тип индекса должен соответствовать оператору в запросе. Индекс не магический: JSONB containment @> может использовать GIN, но произвольная функция над JSON может не использовать индекс без expression index.

Пример: WHERE data @> '{"role": "admin"}' может выиграть от GIN по JSONB, а WHERE lower(email)=... требует либо хранения нормализованного email, либо expression index на lower(email).

91. Что такое covering index / index-only scan?

Ожидаемый ответ: index-only scan возможен, когда БД может ответить на запрос, используя только индекс, без чтения строк таблицы. Для этого нужные колонки должны быть в индексе, а visibility information должна позволять не ходить в heap.

Объяснение: в PostgreSQL даже если все колонки есть в индексе, visibility map влияет на то, можно ли избежать heap fetches. Частые обновления могут снижать пользу index-only scan.

Практический вывод: INCLUDE-колонки могут помочь покрыть запрос, но каждый дополнительный индекс и каждая включенная колонка увеличивают размер индекса и стоимость записи.

92. Что такое partial index и когда он полезен?

Ожидаемый ответ: partial index индексирует только строки, удовлетворяющие условию. Он полезен, когда запросы часто работают с маленьким подмножеством данных: активные записи, pending jobs, не удаленные сущности, конкретный tenant tier.

Пример: CREATE INDEX ON jobs (run_at) WHERE status = 'pending'; помогает worker-ам быстро находить готовые задачи, не индексируя завершенные jobs.

Объяснение: partial index меньше, быстрее обновляется и лучше помещается в cache. Но planner использует его только если условие запроса логически соответствует predicate индекса.

93. Что такое expression index?

Ожидаемый ответ: expression index строится по выражению, а не по сырой колонке: например CREATE INDEX ON users (lower(email));. Он полезен, когда запрос фильтрует или сортирует по вычисляемому выражению.

Объяснение: если написать WHERE lower(email) = lower(\$1), обычный индекс по email обычно не поможет, потому что функция применяется к каждой строке. Expression index позволяет хранить вычисленное значение в индексе.

Риск: expression должен совпадать с запросом, а функции должны быть подходящими для индексации. Также индекс увеличивает стоимость записи.

94. Почему слишком много индексов - проблема?

Ожидаемый ответ: каждый индекс занимает место, должен обновляться при insert/update/delete, увеличивает WAL/redo log, может замедлять bulk operations, усложняет planning и повышает требования к cache.

Объяснение: индекс ускоряет конкретные чтения, но является дополнительной структурой записи. Если добавить индекс "на всякий случай", можно улучшить один редкий запрос и ухудшить частые записи.

Сильный ответ: кандидат предлагает отслеживать unused indexes, slow queries, write amplification и удалять индексы после проверки.

95. Что такое transaction isolation anomaly на примере lost update?

Ожидаемый ответ: lost update возникает, когда две транзакции читают одно значение, независимо считают новое и записывают результат, из-за чего одно обновление затирает другое.

Пример: два запроса одновременно читают баланс 100. Первый списывает 10 и пишет 90. Второй списывает 20, но тоже исходил из 100 и пишет 80. Итог должен быть 70, но получилось 80.

Как избежать: atomic update с условием, row lock FOR UPDATE, optimistic locking по version, serializable isolation с retry. Просто обернуть операции в транзакцию недостаточно, если изоляция и locking не предотвращают гонку.

96. Что такое write skew?

Ожидаемый ответ: write skew - аномалия, когда две транзакции читают общий набор данных, принимают решение, а затем обновляют разные строки так, что нарушается инвариант, хотя прямого конфликта записи в одну строку нет.

Пример: в расписании всегда должен быть хотя бы один дежурный врач. Две транзакции одновременно видят двух врачей на смене и каждая снимает одного разного врача. Обе обновляют разные строки, но итог - ни одного дежурного.

Как избежать: serializable isolation, explicit locks на набор/таблицу/агрегат, материализованный invariant row, constraints там, где возможно. Middle+ должен понимать, что row-level lock одной строки не всегда защищает инвариант над набором строк.

97. Что такое phantom read и почему он важен для бизнес-инвариантов?

Ожидаемый ответ: phantom read возникает, когда повторный запрос по условию внутри транзакции видит новый набор строк из-за вставки/удаления другой транзакции. Это важно, если бизнес-правило зависит от количества или отсутствия строк.

Пример: "у пользователя может быть только один активный subscription". Две транзакции одновременно проверяют, что активной подписки нет, и обе вставляют новую. Без unique constraint или serializable/locking получится нарушение.

Практический вывод: инварианты лучше закреплять constraints/unique indexes, а не только проверками в коде.

98. Чем pessimistic locking отличается от optimistic locking?

Ожидаемый ответ: pessimistic locking блокирует данные до изменения, предполагая высокий риск конфликта: SELECT ... FOR UPDATE. Optimistic locking не блокирует заранее, а проверяет версию при записи, предполагая, что конфликты редки.

Объяснение: pessimistic locking проще для критичных операций, но может снижать concurrency и приводить к deadlocks. Optimistic locking лучше масштабируется при редких конфликтах, но требует retry/конфликтного ответа клиенту.

Критерий выбора: высокая конкуренция за одни и те же строки и строгие деньги/остатки - часто pessimistic или atomic update. Редактирование профиля/документа - часто optimistic version.

99. Что делает `SELECT ... FOR UPDATE`?

Ожидаемый ответ: он читает строки и ставит row-level lock, запрещая другим транзакциям изменить или заблокировать эти строки несовместимым lock до commit/rollback текущей транзакции.

Объяснение: lock действует только на найденные строки. Если нужно защитить отсутствие строки или диапазон, могут понадобиться unique constraints, higher isolation, predicate locks или explicit locking strategy.

Практика: для перевода денег блокируют оба счета в стабильном порядке, например по возрастанию id, чтобы снизить риск deadlock.

100. Что такое deadlock в БД и как его предотвращать?

Ожидаемый ответ: deadlock возникает, когда транзакции ждут locks друг друга по циклу. БД обычно обнаруживает deadlock, прерывает одну транзакцию, и приложение должно корректно обработать ошибку.

Профилактика: брать locks в стабильном порядке, держать транзакции короткими, не делать внешние HTTP-вызовы внутри транзакции, индексировать условия update/delete, избегать интерактивных долгих транзакций.

Сильный ответ: кандидат говорит о retry только для безопасных транзакций и с ограничением попыток/backoff.

101. Почему нельзя делать внешние API-вызовы внутри DB-транзакции?

Ожидаемый ответ: внешние вызовы непредсказуемы по latency и могут зависнуть. Пока транзакция открыта, она держит locks, connection из pool, snapshot/версионность и может блокировать другие операции.

Объяснение: это увеличивает contention и риск deadlock/timeouts. Кроме того, если внешний side effect выполнен, а транзакция потом rollback, возникает рассинхронизация.

Лучший подход: сначала зафиксировать состояние и outbox event в транзакции, затем асинхронно вызвать внешний сервис. Для синхронных сценариев нужно очень явно понимать idempotency и компенсаци.

102. Как правильно использовать `defer tx.Rollback()` в Go?

Ожидаемый ответ: после `BeginTx` часто делают `defer tx.Rollback()`. Если позже `Commit` успешен, deferred rollback вернет ошибку, которую обычно игнорируют. Это гарантирует rollback на любом early return до commit.

Объяснение: важно не затереть ошибку commit и не считать транзакцию успешной до успешного commit. Ошибка commit может означать, что изменения не зафиксированы или статус неизвестен, в зависимости от драйвера/сети.

Пример паттерна: `begin, defer rollback, операции, commit, return`. В сложном коде лучше иметь helper `WithTx(ctx, func(tx) error)`.

103. Как настроить connection pool в Go и какие симптомы неправильной настройки?

Ожидаемый ответ: в `database/sql` важны `SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`, `SetConnMaxIdleTime`. Слишком маленький pool вызывает ожидание connections и рост latency. Слишком большой pool перегружает БД и увеличивает contention.

Объяснение: pool приложения должен соответствовать capacity БД и числу инстансов сервиса. Если 20 pod-ов по 100 connections, БД увидит до 2000 connections, что может быть катастрофой.

Метрики: open connections, in-use, idle, wait count, wait duration, query latency, DB CPU, locks. Middle+ должен связать latency приложения с ожиданием connection pool, а не только с медленными SQL.

104. Что такое prepared statements и parameterized queries?

Ожидаемый ответ: parameterized query отделяет SQL-код от пользовательских значений, защищая от SQL injection и позволяя драйверу/БД безопасно передавать параметры. Prepared statement может быть предварительно разобран/спланирован и переиспользован.

Объяснение: безопасность - главный аргумент. Performance зависит от БД, драйвера и plan caching. Нельзя подставлять user input через string concatenation, особенно в WHERE. Для dynamic identifiers используют allowlist, потому что параметры обычно не заменяют имена таблиц/колонок.

105. Что такое SQL injection в ORDER BY и как от нее защищаться?

Ожидаемый ответ: параметры безопасно подставляют значения, но не имена колонок или направление сортировки. Если пользователь передает `sort=created_at desc; drop table...`, конкатенация в ORDER BY опасна.

Защита: allowlist: map внешних имен сортировки в заранее известные SQL expressions. Direction тоже allowlist ASC/DESC. Не принимать произвольный SQL-fragment от клиента.

106. Почему SELECT * часто плохая идея?

Ожидаемый ответ: SELECT * тянет лишние данные, увеличивает network IO, мешает index-only scans, делает код чувствительным к изменению схемы и может случайно начать читать тяжелые поля.

Объяснение: явный список колонок - это контракт. Он упрощает review, снижает нагрузку и защищает от сюрпризов при добавлении новых колонок вроде JSON/blob.

107. Offset pagination: почему она деградирует?

Ожидаемый ответ: при OFFSET N LIMIT M БД часто должна найти и пропустить N строк, поэтому большой offset становится дорогим. Кроме того, при новых вставках/удалениях между запросами пользователь может увидеть дубли или пропуски.

Лучшее решение: keyset/cursor pagination по стабильному порядку, например (created_at, id). Cursor хранит последнее значение сортировки и id, следующий запрос делает WHERE (created_at, id) < (\$lastCreatedAt, \$lastID).

108. Как проектировать cursor pagination правильно?

Ожидаемый ответ: нужен детерминированный порядок, уникальный tie-breaker, непрозрачный cursor, стабильные фильтры, направление сортировки, обработка deleted/inserted rows. Cursor обычно кодирует значения sort keys, а не номер страницы.

Объяснение: если сортировать только по created_at, несколько строк с одинаковым временем могут пропадать или дублироваться. Поэтому добавляют id как tie-breaker.

Сильный ответ: кандидат учитывает безопасность cursor: не доверять ему слепо, подписывать или валидировать, не раскрывать лишние данные.

109. Как безопасно делать schema migration на большой таблице?

Ожидаемый ответ: избегать долгих exclusive locks. Делать изменения маленькими шагами: добавить nullable column без тяжелого default, задеплоить код с dual write/read fallback, backfill батчами с throttling, добавить constraint/index concurrently, переключить чтение, затем удалить старую колонку отдельным шагом.

Объяснение: опасность не в самой миграции как SQL-файле, а в lock duration, WAL, replication lag, IO и несовместимости старой/новой версии приложения.

Сильный ответ: кандидат спрашивает про конкретную БД и версию, потому что поведение DDL отличается между PostgreSQL/MySQL.

110. Как добавлять NOT NULL колонку с default на большой таблице?

Ожидаемый ответ: зависит от БД и версии. Безопасный общий подход: добавить nullable column, начать писать значение из приложения, backfill старыми строками батчами, добавить constraint/validate, затем сделать колонку NOT NULL.

Объяснение: в некоторых версиях БД добавление column with default может переписать всю таблицу или взять тяжелый lock. Даже если современная версия оптимизирует fast default, нужно проверить поведение.

111. Что такое online index creation?

Ожидаемый ответ: создание индекса без длительной блокировки записей. В PostgreSQL это `CREATE INDEX CONCURRENTLY`, в MySQL - online DDL в зависимости от engine/version. Оно обычно дольше и имеет ограничения, но снижает downtime.

Объяснение: обычный `CREATE INDEX` может блокировать writes. Concurrent index build снижает impact, но все равно создает нагрузку на IO/CPU и может усилить replication lag.

112. Что такое replication lag и чем он опасен?

Ожидаемый ответ: replication lag - задержка между primary и replica. Опасность: приложение читает с replica и не видит только что записанные данные, пользователь получает stale read, фоновые процессы принимают решения на старых данных.

Пример: после создания заказа клиент сразу делает GET, который идет на replica, и получает 404. Это read-after-write inconsistency.

Решения: читать критичные read-after-write запросы с primary, sticky reads на время, отслеживать lag, использовать causal consistency tokens, проектировать UI/flow с eventual consistency.

113. Когда читать с replica, а когда с primary?

Ожидаемый ответ: replica подходит для read-heavy аналитических/некритичных запросов, где stale data допустима. Primary нужен для транзакционных reads, read-after-write, проверок инвариантов, authorization/permissions, финансовых операций.

Сильный ответ: кандидат не переносит все чтения на replica ради "масштабирования", а классифицирует запросы по требованиям консистентности.

114. Что такое read/write splitting и какие у него ловушки?

Ожидаемый ответ: read/write splitting направляет writes на primary, reads на replicas. Ловушки: replication lag, транзакции, read-after-write, разные планы/статистика на replica, failover, сложность отладки.

Практика: слой доступа к данным должен явно уметь выбирать consistency level, а не случайно отправлять любой SELECT на replica.

115. Что такое sharding и когда он нужен?

Ожидаемый ответ: sharding - горизонтальное разделение данных по shard key между несколькими БД/кластерами. Он нужен, когда один кластер уже не справляется по объему, throughput, storage, tenant isolation или operational limits.

Объяснение: sharding резко повышает сложность: cross-shard queries, transactions, rebalancing, hot shards, migrations, operational tooling. До sharding обычно стоит оптимизировать схему, индексы, queries, caching, partitioning, read replicas.

116. Как выбрать shard key?

Ожидаемый ответ: shard key должен равномерно распределять нагрузку, соответствовать частым access patterns, минимизировать cross-shard operations и позволять growth/rebalancing. Часто это tenant_id/user_id, но нужно учитывать hot tenants.

Риск: timestamp как shard key может создать hot shard. Слишком мелкий random key может разрушить locality и усложнить запросы по tenant.

117. Чем partitioning отличается от sharding?

Ожидаемый ответ: partitioning делит таблицу внутри одной логической БД/кластера на partitions, обычно прозрачно для приложения. Sharding делит данные между независимыми БД/кластерами, часто требует routing на уровне приложения/инфраструктуры.

Объяснение: partitioning помогает с управлением большими таблицами, pruning, retention, индексами по частям. Но он не убирает все ограничения одного кластера и не всегда увеличивает write capacity линейно.

118. Когда полезно partitioning по времени?

Ожидаемый ответ: для event/log/audit/time-series таблиц, где большинство запросов ограничено периодом, а старые данные нужно архивировать или удалять. Можно быстро drop old partition вместо массового delete.

Риск: если запросы не фильтруют по partition key, pruning не сработает и придется сканировать много partitions. Слишком много partitions тоже ухудшает planning/maintenance.

119. Как проектировать soft delete?

Ожидаемый ответ: soft delete обычно добавляет deleted_at или status. Нужно решить, участвуют ли удаленные записи в unique constraints, запросах, audit, восстановлении и retention. Часто нужны partial unique indexes, например уникальный email только для deleted_at IS NULL.

Объяснение: soft delete усложняет каждый запрос: нужно не забывать фильтр. Ошибка может показать удаленные данные пользователю. Иногда лучше hard delete + audit/archive, в зависимости от домена и требований.

120. Как хранить audit log?

Ожидаемый ответ: audit log должен быть append-only, содержать actor, action, target, timestamp, correlation/request id, before/after или diff, source/service, metadata. Для критичных изменений audit пишется в той же транзакции, что и бизнес-изменение.

Объяснение: audit нужен для расследований и compliance, поэтому он должен быть надежным и защищенным от тихого изменения. Но нельзя бездумно писать PII/secrets; нужны retention и access control.

121. Что такое outbox pattern и почему он связан с БД?

Ожидаемый ответ: outbox pattern решает проблему атомарности между записью в БД и публикацией сообщения. В одной транзакции с бизнес-изменением приложение записывает outbox row. Отдельный publisher читает outbox и публикует в брокер, затем помечает как отправленное.

Объяснение: без outbox возможны два сбоя: БД commit прошел, событие не отправлено; или событие отправлено, а транзакция rollback. Outbox переводит задачу в надежный retryable процесс.

Сильный ответ: consumers все равно должны быть idempotent, потому что publisher может отправить событие повторно.

122. Что такое inbox/idempotent consumer pattern?

Ожидаемый ответ: consumer сохраняет id обработанного сообщения или idempotency key в БД с unique constraint. При повторной доставке он видит, что сообщение уже обработано, и не выполняет side effect повторно.

Объяснение: брокеры часто дают at-least-once delivery. Это значит, что дубли - нормальное поведение, а не исключение. Идемпотентность consumer-а защищает от повторных списаний, повторных email и дублирующих записей.

123. Как проектировать уникальность с учетом race conditions?

Ожидаемый ответ: уникальность должна быть закреплена unique constraint/index в БД. Проверка "сначала SELECT, потом INSERT" без constraint не защищает от гонки.

Пример: два запроса регистрации одновременно проверяют, что email свободен, оба видят пусто и оба пытаются вставить. Только unique index гарантирует, что один insert пройдет, второй получит conflict.

Практика: application-level pre-check может улучшить UX, но source of truth - constraint. Ошибку unique violation нужно мапить в доменную ошибку.

124. Как хранить деньги и decimal-значения?

Ожидаемый ответ: не использовать float для денег из-за ошибок бинарного представления и округления. Использовать integer minor units (cents) или decimal/numeric тип с явными правилами precision/scale.

Объяснение: финансовые операции требуют детерминированного округления и точности. В Go можно использовать int64 minor units или decimal library, но важно не смешивать валюты и хранить currency code.

125. Как моделировать many-to-many связь?

Ожидаемый ответ: через join table с foreign keys, например user_roles(user_id, role_id), unique constraint на пару, индексы под оба направления запросов. Если у связи есть атрибуты, join table становится полноценной сущностью.

Объяснение: array/json со списком ids может казаться проще, но ломает referential integrity, усложняет запросы, индексы и каскадные изменения.

126. Когда JSON/JSONB в реляционной БД - хорошая идея, а когда плохая?

Ожидаемый ответ: JSONB полезен для редких, гибких, слабо структурированных attributes, metadata, внешних payloads, когда не все поля участвуют в строгих связях. Плохая идея хранить в JSON ключевые доменные поля, по которым нужны constraints, joins, частые фильтры и аналитика.

Объяснение: JSONB снижает миграционную стоимость, но переносит часть схемы в приложение. Это может ухудшить качество данных и усложнить индексацию.

127. ORM vs raw SQL: какие trade-off?

Ожидаемый ответ: ORM ускоряет типовые CRUD, mapping и миграцию моделей, но может скрывать SQL, создавать N+1, сложные неэффективные запросы и привязывать доменную модель к persistence. Raw SQL прозрачен и оптимизируем, но требует больше boilerplate.

Сильный ответ: выбор не религиозный. Для сложных запросов и performance-critical paths часто лучше явный SQL, для простых CRUD может быть ORM/query builder. Важно логировать SQL и тестировать реальные queries.

128. Как диагностировать медленный endpoint: БД или приложение?

Ожидаемый ответ: смотреть distributed tracing или хотя бы latency breakdown: время handler-a, время SQL, ожидание connection pool, внешние API, serialization. В БД смотреть slow query log, EXPLAIN ANALYZE, locks, CPU/IO, cache hit ratio.

Объяснение: endpoint может быть медленным не из-за самого SQL, а из-за ожидания свободного connection, lock wait, N+1 queries, большого JSON response, GC pressure или внешнего API.

129. Что такое lock wait и как его увидеть?

Ожидаемый ответ: lock wait - время ожидания блокировки, которую держит другая транзакция. Его можно увидеть через системные views БД, slow query logs, wait events, monitoring. В PostgreSQL смотрят pg_stat_activity, pg_locks, wait_event.

Объяснение: запрос может быть простым по плану, но висеть из-за блокировки. Поэтому одного EXPLAIN недостаточно для live-инцидента; нужно смотреть активные транзакции и locks.

130. Почему долгие транзакции опасны даже без активной работы?

Ожидаемый ответ: долгая транзакция держит snapshot, может удерживать locks, мешать vacuum/cleanup старых версий строк, увеличивать bloat, занимать connection и блокировать schema changes.

Объяснение: в MVCC БД старые версии строк нельзя очистить, пока они потенциально видимы открытой транзакции. Поэтому "idle in transaction" - серьезная production-проблема.

131. Что такое MVCC?

Ожидаемый ответ: Multi-Version Concurrency Control позволяет транзакциям видеть согласованный snapshot данных, пока другие транзакции пишут новые версии строк. Читатели и писатели меньше блокируют друг друга, но появляются старые версии, vacuum/cleanup и особенности изоляции.

Объяснение: MVCC объясняет, почему read committed видит новые committed данные между statement-ами, почему долгие транзакции мешают cleanup, и почему update фактически создает новую версию строки.

132. Что такое vacuum/bloat в PostgreSQL?

Ожидаемый ответ: update/delete создают dead tuples. Vacuum очищает их, когда они больше не видимы активным транзакциям. Bloat - разрастание таблиц/индексов из-за накопления неиспользуемого пространства.

Объяснение: если autovacuum не успевает или долгие транзакции удерживают старые версии, таблица растет, запросы читают больше страниц, cache хуже работает. Это влияет на latency и storage.

133. Как проектировать retention и удаление большого объема данных?

Ожидаемый ответ: не удалять миллионы строк одним большим DELETE в рабочее время. Использовать partitioning и drop partition, batch deletes с лимитами, throttling, архивирование, monitoring replication lag/WAL/locks.

Объяснение: массовое удаление создает locks, WAL, bloat, нагрузку на replica и vacuum. Для time-series данных partition drop часто намного безопаснее.

134. Что такое cache-aside и какие риски у кеширования данных из БД?

Ожидаемый ответ: cache-aside: приложение сначала читает cache, при miss читает БД и кладет значение в cache. Риски: stale data, cache stampede, invalidation complexity, inconsistent serialization, hot keys.

Объяснение: cache не исправляет плохую модель данных автоматически. Он добавляет consistency trade-off. Нужны TTL, jitter, singleflight/request coalescing, invalidation strategy и metrics hit rate.

135. Что такое cache stampede?

Ожидаемый ответ: cache stampede возникает, когда популярный ключ истекает, и много запросов одновременно идут в БД пересчитывать одно значение.

Решения: TTL jitter, early refresh, singleflight/locking, stale-while-revalidate, background refresh, request collapsing. Для критичных hot keys можно прогреть cache заранее.

136. Как выбрать между нормализацией и денормализацией?

Ожидаемый ответ: нормализация снижает дублирование и аномалии записи, улучшает целостность. Денормализация ускоряет чтение и упрощает read models, но создает риск рассинхронизации и усложняет write path.

Объяснение: правильный выбор зависит от read/write ratio, требований консистентности и сложности запросов. Часто базовая transactional model нормализована, а денормализованные read models строятся асинхронно.

137. Что такое referential integrity и когда нужны foreign keys?

Ожидаемый ответ: foreign keys гарантируют, что ссылка указывает на существующую запись, и задают поведение при delete/update. Они защищают качество данных на уровне БД.

Trade-off: FK добавляют проверки и могут усложнять bulk loads/sharding, но отказ от FK требует надежной компенсации в приложении и monitoring orphan records. Для большинства обычных OLTP-систем FK полезны.

138. Что такое ON DELETE CASCADE и какие риски?

Ожидаемый ответ: ON DELETE CASCADE автоматически удаляет дочерние строки при удалении родителя. Риск - случайное удаление большого дерева данных, долгие locks, неожиданная потеря audit/history.

Практика: использовать осознанно, ограничивать права, иметь soft delete/audit там, где данные критичны, и понимать размер каскада.

139. Как хранить и проверять права доступа в БД-запросах?

Ожидаемый ответ: права лучше учитывать прямо в запросах через tenant_id/user_id/ACL joins, чтобы не загрузить лишние данные и не отфильтровывать их только в памяти. В multi-tenant системе почти все таблицы должны иметь tenant boundary, если нет отдельной физической изоляции.

Объяснение: фильтрация доступа после чтения может привести к утечке данных через баг, logs, cache или side channel. DB-level Row Level Security может помочь, но тоже требует дисциплины и тестов.

140. Какие вопросы задать кандидату по реальному инциденту с БД?

Ожидаемый ответ: попросить описать симптомы, метрики, запросы, locks, connection pool, изменения перед инцидентом, mitigation и postmortem. Сильный кандидат говорит о slow query, bad migration, missing index, lock contention, replication lag, pool exhaustion или vacuum/bloat с конкретным расследованием.

Что слушать: не только "добавили индекс", а как убедились, что причина именно в этом; как безопасно применяли fix; какие alerts/tests/runbooks добавили после.

141. Практический кейс: endpoint GET /orders стал медленным после роста данных. Как расследовать?

Ожидаемый ответ: начать с воспроизведения и метрик: latency percentiles, RPS, error rate, DB time, number of queries per request. Проверить N+1, slow query log, EXPLAIN ANALYZE, наличие индексов под filters/sort, offset pagination, размер response, connection pool wait.

Объяснение: рост данных часто проявляет сразу несколько проблем: offset стал большим, индекс не покрывает WHERE tenant_id AND status ORDER BY created_at, ORM делает N+1 для items, response сериализует слишком много полей.

Хорошее решение: добавить/исправить composite или partial index, перейти на cursor pagination, batch-load related data, ограничить page size, добавить metrics и regression test/benchmark для query plan.

142. Практический кейс: иногда создание пользователя возвращает 500 вместо "email занят". Что проверить?

Ожидаемый ответ: проверить unique constraint на email, обработку unique violation в Go, race между pre-check и insert, нормализацию email, collation/case sensitivity, transaction boundaries.

Объяснение: корректная схема: нормализовать email или использовать case-insensitive type/index, иметь unique constraint, при конфликте мапить DB error в доменную ErrEmailAlreadyExists и HTTP 409. Предварительный SELECT не заменяет constraint.

143. Практический кейс: воркеры периодически берут одну и ту же job дважды. Как исправить?

Ожидаемый ответ: использовать атомарное claim-обновление или locking pattern: SELECT ... FOR UPDATE SKIP LOCKED внутри транзакции, затем update status; либо UPDATE ... WHERE status='pending' ... RETURNING. Нужны lease/visibility timeout для recovery после падения

worker-a.

Объяснение: простая схема "SELECT pending, потом UPDATE" без lock/condition race-prone. Два worker-a могут выбрать одну строку до изменения статуса.

Сильный ответ: кандидат учитывает idempotency обработки job, retry count, next_run_at, dead-letter status и мониторинг queue lag.

144. Практический кейс: после релиза выросло число DB connections. Что делать?

Ожидаемый ответ: проверить настройки pool в сервисе, количество pod-ов, утечки rows (`rows.Close()`), долгие транзакции, новые background workers, retry loops, health checks, connection lifetime. Сравнить DB max connections и суммарный потенциальный pool всех инстансов.

Объяснение: в Go `database/sql` lazy-open, но при нагрузке может открыть до `MaxOpenConns` на каждый процесс. Если лимит не задан, поведение может перегрузить БД.

145. Практический кейс: миграция зависла и заблокировала записи. Как действовать?

Ожидаемый ответ: сначала стабилизировать: понять blocking query/lock, оценить impact, при необходимости cancel/terminate migration, откатить deploy или отключить traffic. Потом разобрать DDL, lock level, размер таблицы, наличие long transactions.

Профилактика: lock timeout для миграций, statement timeout, online/concurrent operations, dry run на staging с похожим объемом, миграции маленькими шагами, мониторинг locks и replication lag.

146. Какие DB-вопросы стоит считать обязательными для Middle+ Go backend?

Ожидаемый ответ: транзакции и rollback/commit, базовые индексы и EXPLAIN, N+1, connection pool, уникальность через constraints, migration safety, isolation/lost update, context timeouts для SQL, обработка DB errors.

Объяснение: Middle+ не обязан быть DBA, но должен писать код, который не ломает данные и не создает очевидных production-проблем. Если кандидат не понимает транзакции, индексы и конкурентные обновления, это серьезный риск для backend-поли.

17.14 Как оценивать ответы

Уровень ниже Middle+

- Знает термины, но не может объяснить последствия в production.
- Предлагает unbounded concurrency, retries без timeout, глобальные переменные и panic для обычных ошибок.
- Путается в context, interfaces, transactions, race conditions.
- Не упоминает тесты, observability и edge cases без подсказки.

Уверенный Middle+

- Объясняет базовые механики Go и типичные ловушки.
- Самостоятельно проектирует bounded concurrency и корректно использует context.
- Понимает транзакции, индексы, миграции и надежную работу с внешними API.
- Пишет тестируемый код и видит эксплуатационные риски.
- Может аргументировать trade-off и коммуницировать спокойно.

Сильный Middle+ / Senior-

- Начинает с требований и ограничений, а не с любимой технологии.
- Видит failure modes: partial failure, duplicate delivery, race, deadlock, overload.

- Предлагает простое решение с понятным путем развития.
- Связывает код, архитектуру и эксплуатацию: metrics, alerts, rollout, incident response.
- Умеет объяснить, какие решения являются обязательными сейчас, а какие можно отложить.